```
    public function calculate(Request $request): JsonResponse
    {
        // Validate request
        $validated = $request->validate([
            'num1' => 'required|numeric',
            'num2' => 'required|numeric',
            'operation' => 'required|string|in:add,subtract,multiply,divide',
        ]);

        try {
            // Create DTO instance
            $dto = new CalculatorDTO($validated['num1'], $validated['num2'], $validated['operati

            // Process calculation
            $result = $this->calculatorService->calculate($dto);

            return response()->json([
                'success' => true,
                'num1' => $dto->num1,
                'num2' => $dto->num2,
                'operation' => $dto->operation,
                'result' => $result
            ], 200);
        } catch (\Exception $e) {
            return response()->json(['error' => $e->getMessage()], 400);
        }
    }
}
```

This controller handles the API logic for the calculator. It validates the input, creates a DTO instance, and calls the `CalculatorService` to perform the calculation. If an error occurs (e.g., division by zero), it returns a JSON error response.

14.11.4.6 Step 6: Set Up the API Routes

Edit the **routes/api.php** file and define API routes:

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CalculatorController;

Route::post('/calculate', [CalculatorController::class, 'calculate']);
```

This creates a single endpoint for the calculator API:

14.11.4.7 Step 7: Test the API with Postman or cURL

After completing the above steps, start the Laravel development server:

```
php artisan serve
```

Next, open Postman or your preferred API testing tool and test the endpoint.